

MAINTAINER PDF EDITION

Artifact Tree Guide

Filesystem-side evidence, rebuild surface, and audit contract for The Ontology Machine.

Core rule: the Corpus DB is the query surface; the Artifact Tree is the inspection and rebuild surface that proves how the DB got there.

Prepared from `06_Artifact_Tree_Guide.md`. Styled for readability, clean tables, code blocks, and rendered evidence-flow diagrams.

Contents

1. Artifact Tree Guide.....	3
2. Contract Source.....	4
3. Canonical Top-Level Layout	5
4. Mental Model.....	6
5. Folder Reference	7
6. Artifact Lifecycle Table.....	14
7. Rebuild Behavior	15
8. Merge Behavior	16
9. Semantic Release Staging.....	17
10. DB-Local Page Images	18
11. What Can Be Deleted	19
12. Inspection Checklist	20
13. Failure Patterns.....	21
14. Handover Rule.....	22

Recommended first pass: read sections 1-4, then use Folder Reference and Inspection Checklist for operational work.

1. Artifact Tree Guide

The Artifact Tree is the filesystem-side evidence and rebuild surface of The Ontology Machine. It is the place where source files, page images, stage outputs, requests, logs, Error Cases, Corpus DBs and Semantic Release packages are kept together under one target root.

The Corpus DB is the queryable materialization. The Artifact Tree is the inspection and rebuild surface that proves how the DB got there.

This distinction matters. A DB row can be updated, reset, rebuilt or merged. The Artifact Tree preserves the surrounding evidence: the original file, the rendered page images, raw extracts, model requests, structured outputs, validation reports, normalized payloads and release package. A maintainer must be able to stand in front of a Corpus DB and answer three questions:

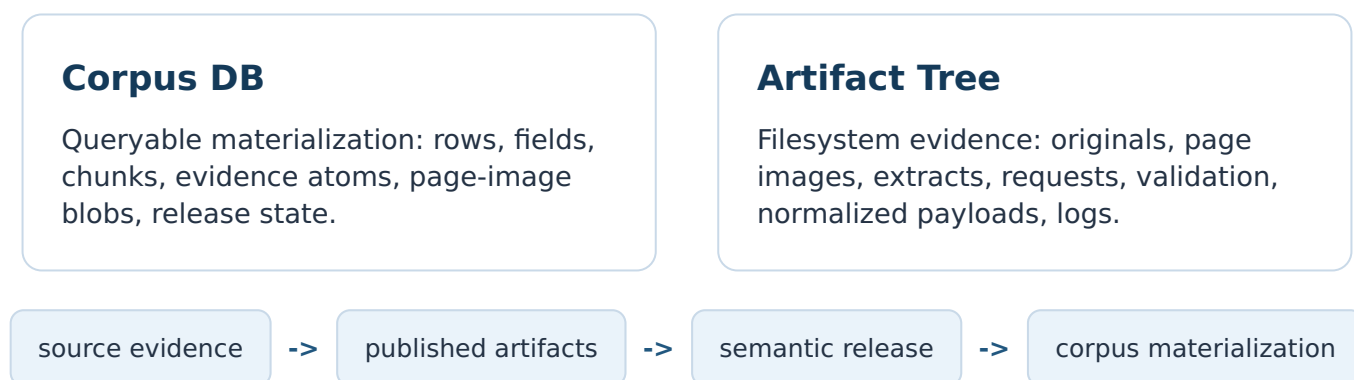


Figure 1. Minimum evidence model: the DB is the query surface; the tree proves how the DB got there.

1. Which source evidence created this row?
2. Which semantic release was active when it was materialized?
3. Can the DB be rebuilt or audited from the artifacts without trusting chat or memory?

The Artifact Tree exists to make the answer yes.

2. Contract Source

The canonical folder contract is implemented by both the Orchestrator and the Corpus Builder.

Primary code anchors:

- 00 - Orchestrator/orchestrator/workspace_domain/types.py
- 00 - Orchestrator/orchestrator/workspace_domain/workflow.py
- 00 - Orchestrator/config/artifact_publication_policy.json
- 05 - Corpus Builder/corpus_builder/standalone_artifacts/artifact_tree_contract.py
- 08 - Semantic Control Kernel/semantic_control_kernel/workflows/database_creation/artifact_tree_contract.py

The current folder contract version is:

```
kernel_artifact_tree.v1
```

Both spelling forms exist in older conversation and filenames. The code and current documentation use **Artifact Tree**.

3. Canonical Top-Level Layout

Every valid Artifact Tree contains these top-level folders:

```
Artifact Tree\  
  Input\  
  Corpus\  
  Semantic Release\  
  Documents\  
  Error Cases\
```

The required `Documents` subfolders are:

```
Artifact Tree\  
  Documents\  
    logs\  
    normalized\  
    originals\  
    page_images\  
    raw_extracts\  
    requests\  
    structured\  
    validation\
```

The full required folder set is:

```
Input  
Corpus  
Semantic Release  
Documents/logs  
Documents/normalized  
Documents/originals  
Documents/page_images  
Documents/raw_extracts  
Documents/requests  
Documents/structured  
Documents/validation  
Error Cases
```

The Orchestrator can create or validate this tree. Validation checks that all required paths exist and are directories. It can also require an empty `Input` folder for workflows where that matters.

4. Mental Model

The Artifact Tree is organized around the document lifecycle.

```
Input
  user drops or selects source files

Documents/originals
  immutable copy of source files after successful publication

Documents/page_images
  immutable visual ground truth for pages

Documents/raw_extracts
  Optimizer output, before semantic judgement

Documents/requests
  persisted OCR, Interpreter and Normalizer request payloads

Documents/structured
  Interpreter output

Documents/validation
  Validator reports

Documents/normalized
  Normalizer output consumed by the Corpus Builder

Corpus
  SQLite Corpus DB and sidecar files

Semantic Release
  taxonomy/projection/release package used by the target DB

Error Cases
  frozen evidence for failed or rejected processing

Documents/logs
  run logs, batch manifests, merge receipts, reset manifests and support traces
```

The folders are intentionally boring. That is a feature. When something fails, the maintainer should not need to reverse-engineer a clever storage layout before finding the evidence.

5. Folder Reference

Input

Input is the user-facing intake surface.

Files placed here are candidates for ingestion. During direct Orchestrator runs or Kernel manual pipeline runs, the system inventories the input set and then routes each supported file through the pipeline.

Input is not the long-term source archive. Once a file has been successfully published, the durable source copy belongs in **Documents/originals**. The input folder may be empty after a run, depending on the workflow and cleanup policy.

Preserve when:

- A run has not started yet.
- A failed run must be retried from exactly the same input set.
- Support needs to prove what the user submitted.

Can usually be cleaned when:

- The run completed.
- Originals are published.
- The batch manifest records the post-run original locations.

Corpus

Corpus contains the SQLite Corpus DB and SQLite sidecars.

Typical contents:

```
Corpus\  
  corpus.db  
  corpus.db-wal  
  corpus.db-shm
```

The DB path must stay inside this folder for Kernel-controlled workflows. A DB outside the artifact tree breaks the target identity model because the Kernel can no longer prove that the selected DB belongs to the selected artifact root.

The Corpus DB is the operational query surface:

- documents
- pages

- chunks
- extracted fields
- extracted rows
- evidence atoms
- embeddings
- Base Graph structures
- ontology lenses
- DB-local page images in `document_page_images`
- Semantic Release snapshot/state

The DB is durable, but it is not the only truth. It can be reset, rebuilt, merged or forked by workflow. The artifact folders preserve the evidence needed to verify or rebuild it.

Preserve when:

- It is the active database.
- It contains ontology lenses or review/correction layers.
- It is a merge source or published sample.

Can be recreated when:

- The Artifact Tree contains the required rebuild artifacts.
- The Semantic Release package is present.
- The rebuild workflow validates normalized payloads against the release.

Semantic Release

`Semantic Release` stores the taxonomy/projection package that defines how documents are normalized and materialized.

The active path used by Kernel creation workflows is:

```
Semantic Release\
  releases\
    <release_id>\
      release.json
```

The release package is not just a label. It carries the semantic contract for the Corpus DB:

- release identity
- release version
- release fingerprint
- taxonomy fingerprint

- projection IDs
- projection fingerprints
- runtime semantic assets needed by Normalizer and Corpus Builder

Ingestion requires an active Semantic Release. Empty database workflows may intentionally stop before release activation, but the pipeline must not materialize new documents without an active release.

Preserve always for audit and rebuild. If the release package is missing, a DB may still contain an active release snapshot, but rebuild and handover become weaker because the filesystem-side semantic evidence is gone.

Documents/originals

`Documents/originals` is the durable source-file archive inside the artifact tree.

Successful publication moves or copies the original evidence here so that later inspection, reingest, merge and handover do not depend on a user's external Downloads/Desktop/input folder.

For merge workflows, original paths are copied additively. If a target path already exists, merge code can namespace the imported artifact below a source database ID or add a stable hash suffix. This prevents source files with the same name from overwriting each other.

Preserve for audit. Delete only if the tree is intentionally being reduced to a DB-only artifact and rebuild from original files is no longer a requirement.

Documents/page_images

`Documents/page_images` contains the rendered page images. These images are the artifact-level visual ground truth.

This is not merely a UI cache. In The Ontology Machine, the image is often the closest durable representation of what the model saw. JSON, normalized fields and DB rows can be wrong, corrected or rebuilt. The page image is the thing a human can inspect directly when asking whether the materialized claim matches the visible source.

Every Corpus DB must also contain page images in the table `document_page_images`. The DB-local images are used for evidence backlinking inside the queryable database and for source/page viewing. The folder images remain the artifact-level rebuild and audit surface.

The two surfaces have different jobs:

Surface	Purpose
<code>Documents/page_images</code>	Filesystem ground truth and rebuild artifact.

Surface	Purpose
<code>document_page_images</code>	DB-local evidence backlinking and viewer retrieval.

Do not describe folder page images as replaceable by DB blobs. The DB copy is important, but the artifact image is what lets rebuild and handover work without trusting the current DB state.

Preserve for audit, rebuild and visual verification.

Documents/raw_extracts

`Documents/raw_extracts` contains Optimizer output.

Raw extracts are intentionally low-semantics. They are meant to preserve locatable text/visual structure without forcing early business, legal, financial or narrative meaning onto the content.

The Corpus Builder rebuild path uses raw extracts as sidecars when available. They are not the rebuild anchor by themselves; normalized artifacts are the primary scan target, and raw extracts help preserve upstream evidence.

Preserve for debugging extraction quality, prompt failures and rebuild traceability.

Documents/requests

`Documents/requests` stores model request payloads that were used during the pipeline.

Current request publication covers:

- `ocr.request.json`
- `interpreter.request.json`
- `normalizer.request.json`

These files matter because they make model behavior inspectable. Without persisted requests, a maintainer can see the model output but not the exact prompt/input shape that produced it. That is not enough for serious debugging.

Multi-page inputs may have nested request folders with budgeted path names to stay below Windows path limits. Do not "prettify" those names by hand. They are short for a reason.

Preserve for debugging, cost analysis, prompt evolution and audit.

Documents/structured

`Documents/structured` contains Interpreter output.

This is the stage where the system records what the model claims to see before the Normalizer maps it into the active taxonomy/projection contract.

Structured artifacts are useful for:

- checking hallucination or omission before normalization
- inspecting raw classification hints
- debugging table/row interpretation
- comparing model output across prompt versions

The rebuild process can use structured files as evidence sidecars when matching normalized outputs.

Documents/validation

`Documents/validation` contains Validator reports.

Validation files explain why a stage output was accepted, marked for review or rejected. The rebuild sidecar resolver chooses validation report suffixes based on the structured artifact profile when possible.

Common suffixes include:

```
.vision_validation_report.json  
.files_validation_report.json  
.validation_report.json
```

Preserve for review flag analysis and failure pattern mining.

Documents/normalized

`Documents/normalized` contains Normalizer output. This is the main rebuild anchor for document materialization.

The Corpus Builder standalone rebuild scanner recursively searches for:

```
*.structured.normalized.json
```

For each normalized file, it tries to find matching sidecars:

- `Documents/structured/.../*.structured.json`
- `Documents/validation/.../*validation_report.json`
- `Documents/raw_extracts/.../*.raw.json`

Then the rebuild workflow validates normalized payloads against the active Semantic Release before loading them into the Corpus DB.

If `Documents/normalized` is missing or incompatible with the release, rebuild cannot be trusted.

Preserve for rebuild.

Documents/logs

`Documents/logs` is the durable operational trace area.

Important subareas include:

```
Documents/logs/  
  pipeline_batches/  
    pbt_<timestamp>_<hash>_<attempt>/  
      pending_pipeline_batch_manifest.json  
      pipeline_batch_manifest.json  
      correlation_report.json  
      batch_run_journal.jsonl  
  resets/  
    rstman_<hash>.json  
  selections/  
    <sample_selection_id>/  
      sample_selection_manifest.json  
  merge_runs/  
    <merge_run_id>/  
    ...
```

The exact file set depends on the workflow, but the rule is stable: `Documents/logs` explains how a workflow moved through the system.

Pipeline batch manifests are especially important. A Kernel manual pipeline run is not complete merely because the Orchestrator process ended. Completion means the final manifest exists, correlation passed, and the Kernel final notice was emitted.

Preserve for support, handover and recovery. Logs can be pruned only with a clear retention policy and only after deciding that recovery/audit history is no longer needed.

Error Cases

`Error Cases` contains frozen evidence for failed or rejected records/pages.

An Error Case is not trash. It is the evidence package for a failed workflow branch. It should contain enough surrounding context to answer:

- Which input failed?

- At which stage did it fail?
- What request/output/report led to the failure?
- Is this an input problem, model problem, taxonomy/projection problem, validation problem or Corpus Builder materialization problem?

Error Cases are useful release evidence. Keeping them in official sample DBs can be intentional because they show what the system writes when something does not pass.

Preserve unless the purpose is to ship a clean demo with no failure evidence.

6. Artifact Lifecycle Table

Artifact	Owner	Mutable?	Rebuild Role	Audit Role	Cleanup Risk
Input/*	User / Orchestrator	Yes	Optional once originals are published	Shows submitted input before publication	Medium
Documents/originals/*	Orchestrator	No in normal operation	Reingest source	Proves source file identity	High
Documents/page_images/*	Optimizer / Orchestrator	No in normal operation	Visual rebuild/audit evidence	Human-checkable ground truth	Very high
Documents/raw_extracts/*	Optimizer	No after publication	Rebuild sidecar	Extraction evidence	High
Documents/requests/*	Orchestrator	No after publication	Debug sidecar	Prompt/model-call evidence	Medium-high
Documents/structured/*	Interpreter	No after publication	Rebuild sidecar	Interpretation evidence	High
Documents/validation/*	Validator	No after publication	Rebuild sidecar	Review/rejection evidence	High
Documents/normalized/*	Normalizer	No after publication	Primary rebuild input	Canonical normalized evidence	Very high
Corpus/*.db	Corpus Builder	Yes through workflows	Rebuild target/source	Queryable materialization	Depends on ontology content
Semantic Release/releases/*/release.json	Normalizer / Kernel route	No after activation	Required release proof	Semantic contract proof	Very high
Documents/logs/pipeline_batches/*	Kernel / Orchestrator	Append/finalize	Recovery/correlation	Workflow proof	Medium-high
Documents/logs/merge_runs/*	Kernel / Corpus Builder	Append/finalize	Merge recovery	Merge provenance	Medium-high
Error Cases/*	Orchestrator	No after bundle	Retry/review evidence	Failure proof	Medium-high

7. Rebuild Behavior

The rebuild workflow turns artifact evidence back into a Corpus DB.

The high-level route is:

1. Resolve the artifact root.
2. Resolve the target DB path under `Corpus`.
3. Load the Semantic Release package.
4. Scan `Documents/normalized` for `*.structured.normalized.json`.
5. For every normalized artifact, resolve matching sidecars from `structured`, `validation` and `raw_extracts`.
6. Validate normalized payloads against the release.
7. Replace existing DB files if explicitly confirmed.
8. Seed the active release snapshot.
9. Load the document bundles.
10. Persist page images into `document_page_images` when configured by the Corpus Builder source settings.

The rebuild scanner can also work with explicit `normalized/structured/ validation/raw` directories, but the Artifact Tree root is the normal handover surface.

Rebuild does not mean "trust whatever JSON exists". The release validation step is what prevents normalized artifacts from being loaded against the wrong taxonomy/projection contract.

8. Merge Behavior

Additive merge workflows create a target Artifact Tree and target Corpus DB from multiple source databases.

The merge system does not treat the artifact folders as a flat copy target. It uses artifact namespaces:

```
Documents/originals
Documents/raw_extracts
Documents/page_images
Documents/requests
Documents/structured
Documents/validation
Documents/normalized
Documents/logs/imported
```

If a source artifact path can be preserved without collision, it is preserved. If the same relative path already exists in the target, merge can place the artifact below a source database ID namespace or add a stable hash suffix.

This is why a merged tree may contain paths that look less pretty than a fresh single-source ingest. Those paths are collision-safe provenance, not random folder noise.

The filled-database merge path also copies DB rows, materialization references, Base Graph structures and ontology lenses. Page image rows in `document_page_images` are part of the document-ID table set that must survive merge.

Important merge logs and receipts belong under:

```
Documents/logs/merge_runs/
```


9. Semantic Release Staging

Kernel creation workflows use the `Semantic Release` folder as the release package target.

Default release route:

```
Semantic Release/  
  releases/  
    semantic_release.default/  
      release.json
```

Custom release route:

```
Semantic Release/  
  releases/  
    <custom_release_id>/  
      release.json
```

Incomplete creation routes may stage taxonomy or projection state without producing an active release yet. In that case the database is not ready for ingest. The correct continuation is a Kernel resume path, not manual folder editing.

10. DB-Local Page Images

The Corpus DB contains:

```
document_page_images(  
    document_id,  
    page,  
    content_type,  
    byte_size,  
    image_sha256,  
    image_blob  
)
```

For V1 databases, this table is part of the expected evidence model. It lets the Query Agent, source viewer and evidence backlinking work from the DB without depending on loose file lookup.

That does not make `Documents/page_images` optional. The folder is the artifact-level ground truth and rebuild surface. The DB table is the query/viewer-local evidence copy.

If these disagree, treat it as an integrity issue:

- folder image missing, DB image present: DB can still view evidence, but rebuild/audit is weakened.
- folder image present, DB image missing: rebuild can repair DB-local evidence, but current source viewing/evidence backlinking is incomplete.
- hashes disagree: inspect the original artifact and materialization path before trusting either surface.

11. What Can Be Deleted

Use this as a conservative field policy.

Usually safe to delete after a completed, backed-up run:

- temporary process logs outside the Artifact Tree
- stale `*.db-wal` and `*.db-shm` only when the DB is closed
- empty folders that are not part of the required contract
- duplicate external input files after originals are published

Delete only with intent:

- `Input` contents after publication
- old pipeline batch logs
- old merge receipts
- Error Cases from demos where failure evidence should not ship

Do not delete in handover or audit trees:

- `Documents/originals`
- `Documents/page_images`
- `Documents/normalized`
- `Semantic Release`
- active Corpus DB
- ontology-bearing Corpus DBs
- current batch manifests and correlation reports

Never clean by guessing from file age alone. A six-month-old Semantic Release package may be more important than a fresh temporary log.

12. Inspection Checklist

When receiving an Artifact Tree from someone else, check it in this order:

1. Required folder contract exists.
2. `Corpus` contains the expected DB.
3. The DB path is actually under the Artifact Tree's `Corpus` folder.
4. `Semantic Release/releases/.../release.json` exists for the active release.
5. `Documents/normalized` contains normalized artifacts if rebuild is expected.
6. `Documents/page_images` exists and is populated for visual evidence.
7. The DB contains `document_page_images`.
8. `Documents/logs/pipeline_batches` contains the final manifest for the last Kernel manual ingest, if the run came through Kernel.
9. `Error Cases` is checked before declaring the dataset clean.
10. Merge trees have `Documents/logs/merge_runs` and source provenance, if they were produced by additive merge.

13. Failure Patterns

Missing active Semantic Release

The DB may exist, but ingestion cannot start. Creation routes can intentionally leave a DB in `no_semantic_release` or `semantic_release_incomplete` state. Resume the Kernel workflow or activate a release before ingest.

DB outside artifact tree

Kernel workflows reject this because the target cannot be proven as the selected artifact tree's Corpus DB.

Normalized artifacts without matching release

Rebuild should stop. Loading normalized JSON against the wrong Semantic Release would create a false DB.

Page images only in folder or only in DB

The system may still partially work, but evidence integrity is incomplete. Repair through rebuild or targeted Corpus Builder maintenance rather than pretending the missing surface does not matter.

Pretty merge cleanup broke provenance

Merged artifact paths may be namespaced or hash-suffixed. Do not flatten them after the fact. If a path looks ugly but collision-safe, it is probably doing its job.

14. Handover Rule

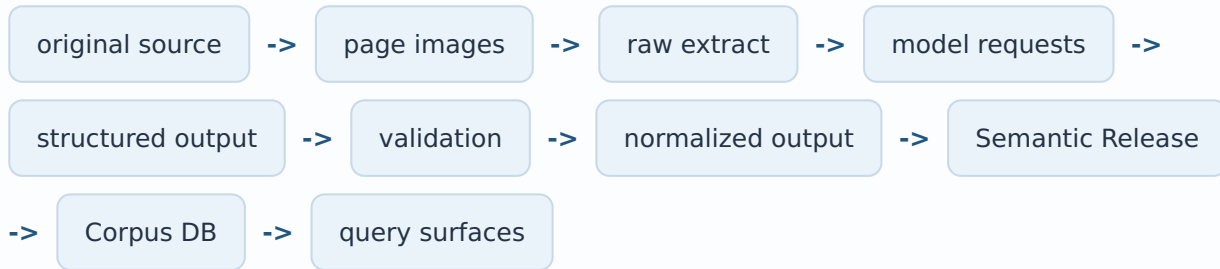


Figure 2. Handover evidence chain: preserve enough artifacts to prove, inspect, rebuild, and query.

A handover-ready Artifact Tree is not the smallest possible folder. It is the smallest folder that still lets another maintainer prove, inspect, rebuild and query the Corpus DB without asking the original operator what happened.

For The Ontology Machine, that means preserving the evidence chain:

```
original source
-> page images
-> raw extract
-> model requests
-> structured output
-> validation
-> normalized output
-> Semantic Release
-> Corpus DB materialization
-> Base Graph / ontology / query surfaces
```

If that chain is intact, the Artifact Tree is doing its job.